# LeafLens

AI-Powered Leaf Disease Prediction System

## High-Level Design (HLD)

**Prepared by**

Abhijeet Kumar

DA24B001

## 1 Introduction

LeafLens is designed to make plant leaf disease classification accessible through a simple image upload interface. A user submits a photograph of a leaf and receives a predicted disease category with its confidence.

The challenge was not just to develop an accurate classifier. We also needed a system that could prepare data consistently, track experiments, serve predictions through an API, and provide visibility into its behaviour after deployment.

This document describes the major design decisions made during development and explains how the different parts of LeafLens work together.

### 1.1 Design Goals

The design follows a few practical goals:

- Keep the user interface separate from model inference.
- Make the data preparation and training process reproducible.
- Use clear interfaces between application modules.
- Track model experiments and their results.
- Handle invalid uploads without unnecessarily invoking the model.
- Monitor the application after deployment.
- Keep local deployment straightforward.

## 2 System Design Overview

LeafLens is organised into four logical areas:

1. **Data and ML development:** dataset management, ingestion, preprocessing, training, and evaluation.
2. **Model serving:** loading the trained checkpoint and performing inference.
3. **User application:** receiving images and presenting predictions.
4. **Monitoring and deployment:** collecting metrics and running the application services.

The offline development workflow and the online prediction workflow are kept separate. This allows model training to be repeated without changing the frontend or the API contract.

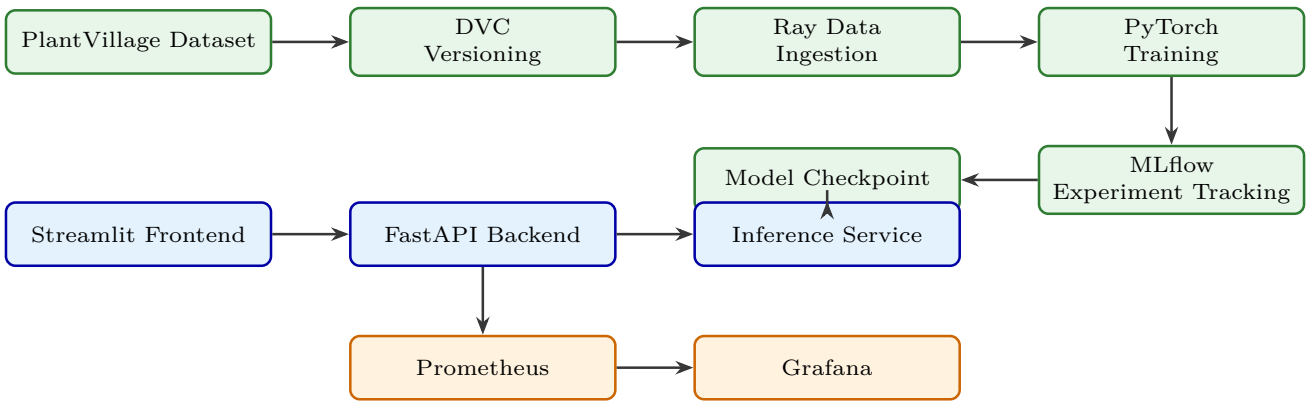



**Figure 1:** High-level design of LeafLens

## 3 Technology Selection and Rationale

The technology choices were based on the requirements of the project, the available development environment, and the need to demonstrate a complete ML application workflow.

**Table 1:** Technology choices

| Area | Technology | Reason for selection |
|---|---|---|
| Model development | PyTorch | Provides flexibility for training and fine-tuning the image classifier. |
| Image model | MobileNetV3-Large | Offers a practical balance between classification capability and model efficiency. |
| Dataset tracking | DVC | Helps maintain dataset and pipeline reproducibility. |
| Data ingestion | Ray Data | Supports scalable data loading and validation. |
| Experiment tracking | MLflow | Records parameters, metrics, runs, and model artifacts. |
| Backend API | FastAPI | Provides a structured REST interface for image prediction. |
| Frontend | Streamlit | Allows a functional upload interface to be built with Python. |
| Metrics collection | Prometheus | Collects application metrics over time. |
| Monitoring dashboard | Grafana | Makes runtime metrics easier to inspect. |
| Deployment | Docker Compose | Runs the application and monitoring services together. |

### 3.1 Why a Separate Backend?

It would have been possible to load the model directly inside the Streamlit application. However, that would mix user-interface code with model-serving logic.

Instead, LeafLens uses FastAPI as a separate backend. The frontend sends an HTTP request and receives a structured response.

This approach gives us a cleaner boundary between the two components. It also makes it possible to test the API independently of the user interface.

### 3.2 Why Prometheus and Grafana?

A successful prediction does not tell us whether the application is healthy over time. We also need to know whether requests are failing, whether latency is increasing, and whether predictions are being generated.

Prometheus collects these operational metrics, while Grafana presents them in a dashboard. This gives the project an observability layer beyond basic model accuracy.

## 4 Data and Model Design

### 4.1 Dataset Organisation

The PlantVillage dataset contains 54,305 images across 38 classes. We use stratified splitting to create training, validation, and test subsets.

**Table 2:** Dataset organisation

| Subset | Number of images |
|---|---:|
| Training | 38,013 |
| Validation | 8,146 |
| Testing | 8,146 |
| Total | 54,305 |

The split is created with random state 42. Stratification helps preserve the representation of classes across the subsets, which is particularly useful because the dataset is not perfectly balanced.

### 4.2 Model Design

MobileNetV3-Large with pretrained ImageNet weights is used as the image classification backbone. The classification layer is adapted for the 38 target classes.

Images are resized to $224 \times 224$ pixels and normalised using the ImageNet preprocessing values during training and inference.

**Table 3:** Model configuration

| Setting | Value |
|---|---|
| Model | MobileNetV3-Large |
| Input size | $224 \times 224$ |
| Batch size | 32 |
| Training epochs | 5 |
| Optimizer | AdamW |
| Learning rate | 0.0001 |
| Weight decay | 0.0001 |
| Loss | CrossEntropyLoss |
| Output categories | 38 |

The trained checkpoint is saved and later loaded by the inference component. This avoids retraining the model whenever the application starts.

## 5 Offline ML Pipeline Design

The training workflow is treated as a separate process from online prediction. It begins with the raw dataset and ends with an evaluated model checkpoint.

DVC tracks the dataset and pipeline stages. Ray Data is used for ingestion and validation, and MLflow records the training experiment.

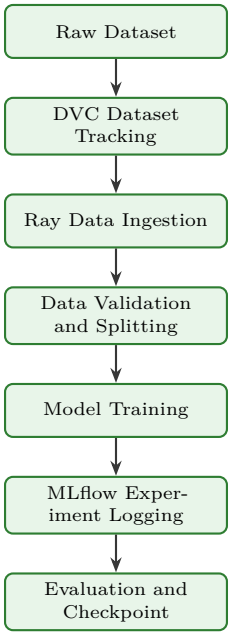



**Figure 2:** Offline ML pipeline

### 5.1 Data Ingestion Design

Ray Data processes the image records and validates the dataset. The completed run processed all 54,305 images, with no invalid images reported.

**Table 4:** Ray Data execution summary

| Metric | Value |
|---|---|
| Images processed | 54,305 |
| Valid images | 54,305 |
| Invalid images | 0 |
| CPU workers | 2 |
| Blocks | 8 |
| Batch size | 128 |
| Execution time | 7.553 seconds |
| Throughput | 7,189.73 images/second |

The ingestion stage also creates a Parquet metadata artifact. This provides a compact record of the processed image information.

### 5.2 Experiment Tracking Design

MLflow is used to keep the training history organised. Parameters, epoch-level metrics, evaluation results, and model artifacts are logged under the LeafLens experiment.

This is useful when comparing training runs or revisiting the configuration associated with a particular checkpoint.

## 6 Online Application Design

The online application follows a simple request-response pattern. The frontend sends the image to the backend, and the backend handles validation and inference.

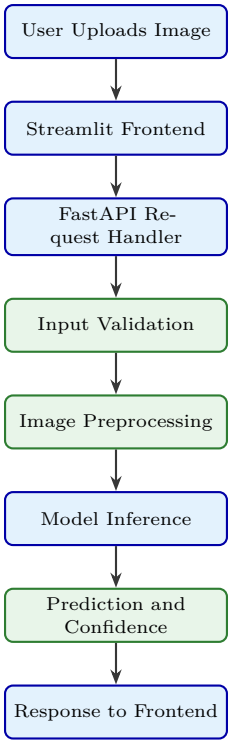



**Figure 3:** Online prediction request flow

### 6.1 Request Handling

The backend checks the uploaded image before calling the model. It rejects empty uploads, files larger than 10 MB, and invalid image content.

Valid images are converted to RGB and passed to the prediction service. The service applies the required transformations and performs classification.

The result is returned using the project's structured Pydantic response schema.

### 6.2 API Design at a Glance

The current backend exposes the following endpoints.

**Table 5:** Backend API overview

| Method | Endpoint | Purpose |
|---|---|---|
| GET | `/` | Basic application endpoint. |
| GET | `/health` | Checks backend health. |
| POST | `/predict` | Accepts an image and returns a prediction. |
| GET | `/metrics` | Exposes Prometheus metrics. |

The detailed request and response specifications are documented separately in the Low-Level Design.

## 7 Monitoring and Deployment Design

### 7.1 Monitoring Flow

The backend exposes application metrics. Prometheus collects them at regular intervals, and Grafana visualises the results.




**Figure 4:** Monitoring data flow

The dashboard includes request rate, backend availability, prediction counts, confidence, P95 latency, HTTP errors, and prediction failures.

### 7.2 Container Deployment

Docker Compose manages four services:

**Table 6:** Container deployment design

| Service | Purpose | Port |
|---|---|---|
| Backend | FastAPI inference API | 8001 |
| Frontend | Streamlit user interface | 8501 |
| Prometheus | Metrics collection | 9092 |
| Grafana | Monitoring dashboard | 3002 |

The frontend communicates with the backend through the internal Docker service address. The backend health check is used to confirm that the API is ready before the frontend starts.

This arrangement keeps the four services separate while allowing them to operate as one application.

## 8 Key Design Decisions

The following decisions had the greatest influence on the design of LeafLens.

**Table 7:** Design decisions and their rationale

| Decision | Reasoning |
|---|---|
| Separate UI and API | Keeps user interaction independent of model-serving code. |
| REST communication | Provides a clear contract between the frontend and backend. |
| Pretrained model | Uses learned image features as a starting point for fine-tuning. |
| DVC and MLflow | Make data processing and experiment history easier to reproduce. |
| Prometheus and Grafana | Provide visibility into runtime behaviour, not just test accuracy. |
| Docker Compose | Simplifies running and demonstrating the complete application. |

## 9 Limitations and Future Scope

The current system has been developed and evaluated using the PlantVillage dataset. Although the test results are strong, the dataset does not represent every condition encountered in real agricultural environments.

Some practical improvements that could be explored in future versions include:

- Evaluating the model on photographs captured under varied lighting and background conditions.
- Adding model-drift monitoring and data-quality checks.
- Improving deployment scalability for multiple simultaneous users.
- Expanding the application with additional plant species and disease categories.

These are possible extensions rather than features claimed by the current implementation.

## 10 Conclusion

The HLD of LeafLens is built around a simple idea: each part of the application should have a clear job.

The ML pipeline prepares and trains the model. The backend handles prediction requests. The frontend makes the system accessible to users, while the monitoring stack helps us understand how it behaves after deployment.

This separation keeps the system organised and makes individual components easier to test, improve, and maintain.